🏆 The Ultimate 2-SAT Pattern Recognition Guide

🧠 The Core Philosophy: "Actors vs. Victims"

Every 2-SAT problem is essentially a conf lct between **Actors** and **Victims**.

1. **Actors (Variables):** The independent entities you control. They MUST have exactly TWO states (e.g., True/False, Flipped/Not Flipped, Taken/Rejected). These are your N variables.
2. **Victims (Clauses):** The constraints or conditions in the problem that *demand* a certain outcome from the actors.

To solve any 2-SAT problem, identify the victims, see which two actors af ect them, and translate their demands into one of the following standard patterns using your API.

📚 The 8 Golden Patterns of 2-SAT

Pattern 1: The "At Least One" (Standard OR)

- **The Situation:** You are forced to choose at least one of two options to satisfy a condition.
- **Example Problem:** *CSES - Giant Pizza* (A person wants either topping A included or topping B excluded).
- **The Logic:** If one fails, the other MUST succeed.
- **The Code:**

```
// "Either A must be TRUE or B must be FALSE"
ts.add_or(A, true, B, false);
```

Pattern 2: The "Mutual Exclusion" (Conf lct)

- **The Situation:** Two elements hate each other. You can choose A, or B, or neither, but you **CANNOT choose both**.
- **Example Problem:** Scheduling conf lcts (Event A and Event B overlap in time).
- **The Logic:** At least one of them must be rejected (False).
- **The Code:**

```
// "A and B cannot be both true -> At least one must be FALSE"
ts.add_or(A, false, B, false);
```

Pattern 3: The "Dependency" (Implication)

- **The Situation:** If you make a specific choice for A, you are forced to make a specific choice for B.
- **Example Problem:** Prerequisite tasks (If I buy item A, I MUST buy item B to make it work).
- **The Logic:** Either I don't buy A (condition dodged), or I buy B (condition satisfied).
- **The Code:**

```
// "If A is TRUE, B MUST be TRUE"
// Translation: Either A is FALSE, or B is TRUE
ts.add_or(A, false, B, true);
```

Pattern 4: The "Opposite Action" (Strict XOR)

- **The Situation:** Exactly ONE of the two actors must perform the action. If one does it, the other cannot. If one doesn't, the other must.
- **Example Problem:** *Codeforces 776D - The Door Problem* (A door is closed, and it's connected to 2 switches. You must press exactly ONE switch to open it).
- **The Logic:** They must end up with different boolean states.
- **The Code:**

```
// "A and B must have different actions" (One pressed, one not)
ts.add_xor(A, true, B, true);
```

Pattern 5: The "Identical Action" (Strict XNOR)

- **The Situation:** The two actors must do the EXACT SAME thing. Either both act, or neither acts.
- **Example Problem:** *Codeforces 1475F - Unusual Matrix* (A cell is already matching the target. You must either flip both its row and column, or flip neither).
- **The Logic:** They must end up with the same boolean state. We enforce this by saying "A is True XOR B is False".
- **The Code:**

```
// "A and B must do the exact same action"
ts.add_xor(A, true, B, false);
```

Pattern 6: The "Forced State" (Unary Constraint)

- **The Situation:** A condition dictates that a variable MUST strictly be in a specif ċ state, otherwise the system fails.
- **Example Problem:** Equations evaluating to a boundary (e.g., A + B = 2, which means both A and B must be 1).
- **The Logic:** Force the value in the graph.
- **The Code:**

```
// "A MUST be TRUE, and B MUST be FALSE"
ts.force_value(A, true);
ts.force_value(B, false);
```

Pattern 7: The "Majority Vote" (2 out of 3 Rule)

- **The Situation:** You have 3 variables, and at least 2 of them must satisfy a certain state.
- **Example Problem:** *Codeforces 1971H - ±1* (A column has 3 numbers, at least 2 of them must be assigned a positive sign).
- **The Logic:** If at least 2 must be positive, it is IMPOSSIBLE for any pair to be simultaneously negative. We add an OR condition for every possible pair.
- **The Code:**

```
// "Out of A, B, C, at least two must be TRUE"
// Therefore, no two can be false at the same time.
ts.add_or(A, true, B, true);
ts.add_or(B, true, C, true);
ts.add_or(C, true, A, true);
```

Pattern 8: The "DAG / Reachability Conf lct"

- **The Situation:** You are selecting edges or nodes to form a single valid path. If two items cannot reach each other, they cannot be chosen together.
- **Example Problem:** *Gym 102412H - Mex on DAG* (Choosing edges to maximize Mex on a simple path).
- **The Logic:** Precompute reachability. For every pair (i, j), if i cannot reach j AND j cannot reach i, they are mutually exclusive (Pattern 2).
- **The Code:**

```
if (!reach[i_end][j_start] && !reach[j_end][i_start]) {
    // They cannot be on the same path, at least one must be discarded
    ts.add_or(i, false, j, false);
}
```

🚀 Final Advice for Competitions

Whenever you see a problem with N ≤ 10^5 asking for a **"Valid Assignment"**, **"Yes/No conf ġuration"**, or **"Binary choices (0/1, f lp/not f lp)"** with pairs of dependencies, immediately pull out your 2-SAT template. Stop thinking about graphs; think strictly in `Actors` and `Victims`.